Lógica Combinacional y Secuencial

Casa Inteligente con Control de Ambientes y Ventilador Automatizado.

PONDERACIÓN: 30

Horas Aproximadas: 40

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Competencias

- Aplicar principios de organización computacional y sistemas embebidos en el diseño e implementación de un sistema de automatización residencial, utilizando microcontroladores, memoria no volátil (EEPROM) y comunicación serial/inalámbrica para gestionar estados digitales persistentes.
- Diseñar e implementar un sistema de control inteligente con configuración personalizable, integrando la carga de escenas predefinidas desde un archivo externo (.org), su almacenamiento en memoria no volátil y su activación remota mediante dispositivos móviles, reforzando el manejo de datos estructurados y validación de comandos.
- Integrar dispositivos electrónicos de control (LEDs, motor DC, LCD, Bluetooth) con lógica digital programada, coordinando salidas digitales, indicadores visuales y retroalimentación en tiempo real para simular un entorno domótico funcional y seguro.

Objetivos

General

Diseñar e implementar una maqueta funcional de una casa inteligente que permita configurar y activar escenas luminosas predefinidas mediante un archivo .org, almacenadas en memoria EEPROM del Arduino, y controlar dispositivos como luces y un ventilador mediante comandos locales o inalámbricos, demostrando la integración práctica de microcontroladores, gestión de memoria y comunicación digital en sistemas automatizados.

Específicos

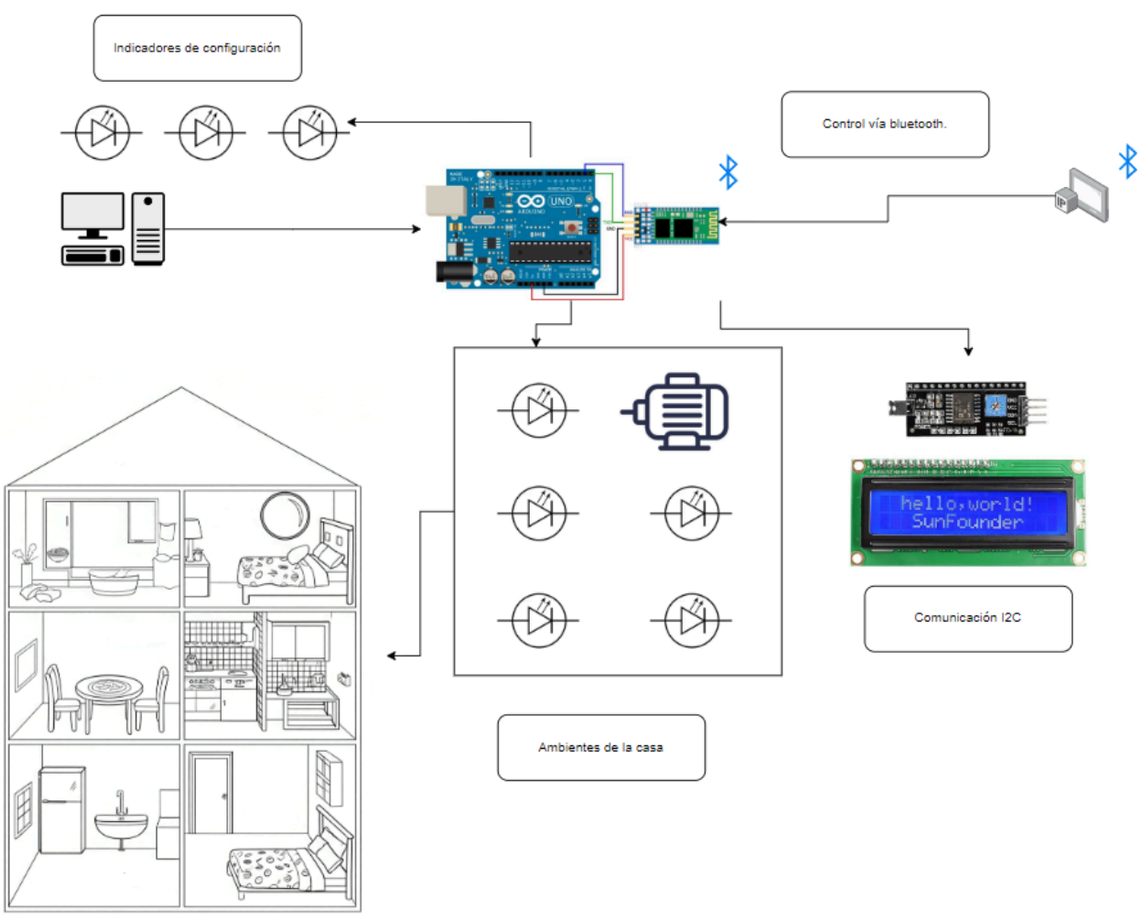
- Implementar un sistema de almacenamiento de escenas luminosas, utilizando la memoria EEPROM interna del Arduino, garantizando persistencia tras reinicios y permitiendo la recuperación de configuraciones sin intervención manual.
- Diseñar una interfaz de configuración que permita cargar escenas desde una PC mediante un archivo con extensión .org, validando su sintaxis y procesando los comandos para actualizar el estado de los ambientes y el ventilador.
- Integrar un módulo Bluetooth para permitir el control remoto de los ambientes desde un dispositivo móvil, ejecutando comandos y proporcionando retroalimentación visual mediante una pantalla LCD y LEDs de estado.

Descripción / Enunciado

En las viviendas convencionales, el control de la iluminación y los dispositivos eléctricos suele realizarse de forma manual e independiente, lo que limita la eficiencia energética, la comodidad y la personalización del ambiente. Este proyecto busca superar estas limitaciones mediante el diseño e implementación de una maqueta funcional de una casa inteligente, donde se integra el control centralizado de luces por ambientes, un sistema automatizado de ventilación y un mecanismo de apertura de puerta mediante servomotor, todo gestionado desde una interfaz remota.

La maqueta representa una vivienda con cinco áreas: sala, comedor, cocina, baño y habitación, cada una equipada con LEDs que simulan sistemas de iluminación. En la habitación, se incluye un motor DC que simula un ventilador de techo, y en la entrada, un servomotor conectado a un Arduino que permite abrir y cerrar la puerta mediante un mismo botón.

El sistema permite configurar escenas luminosas predefinidas (por ejemplo: “Noche”, “Fiesta”, “Despertar”) que se almacenan en memoria EEPROM del Arduino, garantizando persistencia tras cortes de energía. Las escenas pueden activarse remotamente vía Bluetooth, y las led’s operan según la secuencia temporal programada, mejorando la experiencia de usuario y demostrando la integración entre lógica digital y control físico. La incorporación del servomotor permite automatizar la apertura de la puerta, integrando seguridad y comodidad al control centralizado del hogar inteligente.



Entregables

1. Módulo Físico de Casa (Maqueta)

- Estructura física construida en cartón, madera o plástico que representa una vivienda con 5 ambientes claramente identificados:
    - Sala
    - Comedor
    - Cocina
    - Baño
    - Habitación (con espacio para el motor DC como ventilador)
- Espacios para LEDs: Cada ambiente tiene asignado 1 a 4 LEDs que simulan luces de techo o focos.
    - Ejemplo: Sala = 2 LEDs, Cocina = 3 LEDs, Habitación = 1 LED + motor DC.
    - Los LEDs deben estar visibles y etiquetados por ambiente.
- Motor DC como ventilador:
    - Simula un ventilador de techo en la habitación.
    - Gira únicamente en un sentido (no requiere puente H ni inversión de polaridad).
    - Controlado directamente por una salida digital del Arduino (sin transistores adicionales si el motor es pequeño; de lo contrario, usar transistor NPN como interruptor simple).
    - El motor se activa/desactiva según el modo seleccionado.
- Servomotor como puerta:
    - Simula una puerta para permitir o denegar la entrada.
    - El motor se activa/desactiva según el modo seleccionado.

2. Matriz de Iluminación (5 Zonas)

- Cada ambiente (sala, comedor, cocina, baño, habitación) tiene un grupo de LEDs controlables individualmente.
- Total: mínimo 5 LEDs, máximo 16 LEDs (dependiendo de la complejidad deseada).
- Funcionalidad:
    - Encendido/apagado individual por ambiente.
    - Activación simultánea por escenas predefinidas (modos: Fiesta, Relajado, Noche, All).

3. Arduino (Uno/Nano/Compatible)

Unidad central de control que debe realizar las siguientes funciones:

- Gestiona la EEPROM interna para almacenar hasta 4 escenas luminosas (modos).
- Procesa comandos recibidos por:
    - Puerto serial (USB) → para cargar archivos .org desde PC.
    - Bluetooth (HC-06) → para activar modos y leds desde dispositivo móvil.
- Activa salidas digitales para:
    - Encender/apagar LEDs por ambiente.
    - Activar/desactivar el motor DC (ventilador).
- Maneja la pantalla LCD I2C para mostrar estado del sistema.
- Maneja el módulo Bluetooth HC-06 para comunicación inalámbrica.
- Controla el motor DC mediante señal digital (sin puente H, solo un transistor NPN si es necesario).

4. Módulo Bluetooth (HC-06)

- Permite control remoto inalámbrico desde un dispositivo móvil (smartphone/tablet).

- Interfaz móvil: Aplicación terminal (ej: Serial Bluetooth Terminal) que envía comandos para activar modos.
- Comandos soportados (enviados desde app móvil):
    - modo_fiesta
    - modo_relajado
    - modo_noche
    - encender_todo
    - apagar_todo
    - estado

5. Pantalla LCD (I2C, 16x2)

Muestra información en tiempo real al usuario. Debe visualizar:

Modo	Mensaje en LCD
modo_fiesta	Modo: FIESTA. Ventilador: ON, LED'S: Alternandose
modo_relajado	Modo: RELAJADO. Ventilador: OFF, LED'S: OFF
modo_noche	Modo: NOCHE. Ventilador: OFF, LED'S: OFF
encender_todo	LED'S: ON. Ventilador: ON.
apagar_todo	LED'S: OFF. Ventilador: OFF.
Error	ERROR: Modo inválido

6. LED's de Estado (Indicadores Visuales)

Se utilizan 3 LEDs para retroalimentación visual durante la configuración y operación:

LED	Color	Mensaje en la LCD	Función
L1	Azul	Sistema activo	Encendido cuando el sistema está listo para recibir comandos.
L2	Verde	Éxito en configuración	Parpadea o enciende brevemente cuando se carga un archivo.org.
L3	Rojo	Error	Encendido permanente o parpadeante si hay error en el archivo.org o fallo en la EEPROM.

7. Ventilador (Motor DC)

- Ubicación: Dentro de algún ambiente dentro de la maqueta.
- Funcionamiento:
    - Gira únicamente en un sentido.
    - Se activa/desactiva mediante una señal digital del Arduino.

- Control:
    - Activado en modos: fiesta y encender_todo.
    - Desactivado en modos: relajado, noche y apagar_todo.

8. Puerta (Servomotor):

Para complementar el sistema de automatización de la casa, se integrará un servomotor controlado mediante Arduino para accionar la apertura y cierre de la puerta principal. Un mismo botón conectado al Arduino permitirá alternar entre abrir y cerrar la puerta: al presionarlo, el Arduino enviará la señal adecuada al servomotor para mover su eje al ángulo correspondiente, liberando o asegurando el pestillo de la puerta.

- **Ubicación**: Entrada principal de la maqueta (puerta de acceso).
- **Funcionamiento**: El servomotor gira en ambos sentidos según sea necesario:
    - Gira en un sentido para abrir la puerta.
    - Gira en el sentido contrario para cerrar la puerta.
    - Se activa mediante una señal digital del Arduino.
    - Un mismo botón permite alternar entre abrir y cerrar la puerta.
    - El movimiento se detiene automáticamente al alcanzar los ángulos límite predefinidos para apertura y cierre.
- **Control:**
    - Activado:
        - Al presionar el botón para abrir la puerta.
    - Desactivado:
        - Cuando la puerta llega a la posición cerrada.
        - Al presionar nuevamente el botón para cerrar la puerta.

9. Interfaz de Configuración (Archivo .org)

Permite definir los estados de luces y ventilador para cada modo desde una computadora.

Ejemplo del archivo:

```
// Esto es un comentario
// Configuración de modos para el sistema de control

conf_ini // validar que venga si no mostrar error

// Modo FIESTA
modo_fiesta // el modo siempre va a venir de esta manera para evitar algun inconveniente con los modos del telefono
Mensaje en LCD: "Modo: FIESTA." // Esto puede o no venir ya que el modo se define arriba
Ventilador: ON // El otro modo es OFF
LED’S: Alternandose // La forma de Alternandose debe de encender las led's de forma intermitente en varios ambientes

conf:fin // validar que venga si no mostrar error
```

// Esto es un comentario
// Configuración de modos para el sistema de control

conf_ini // validar que venga si no mostrar error

// Modo FIESTA
modo_fiesta
//el modo siempre va a venir de esta manera para evitar algún inconveniente con los modos del teléfono

Mensaje en LCD: "Modo: FIESTA." // Esto puede o no venir ya que el modo se define arriba

Ventilador: ON // El otro modo es OFF

LED’S: Alternandose

// La forma de Alternándose debe de encender las led's de forma intermitente en varios ambientes

conf:fin // validar que venga si no mostrar error

Proceso de Carga:

- El archivo se envía vía puerto serial (USB) desde una PC usando un programa como Arduino IDE, Putty o una app Python.
- El Arduino lee línea por línea, interpreta los comandos y guarda los estados en EEPROM.
- Al finalizar, el LED Verde parpadea 3 veces y el LCD muestra: "Configuracion guardada".

Validación: Si el archivo tiene errores de sintaxis (ej: salon:on en lugar de sala:on), el LED Rojo se enciende y el LCD muestra: "Error en archivo .org".

10. Documentación (Entrega Obligatoria)

Debe incluir:

- Carátula (nombre del proyecto, grupo, integrantes, fecha)
- Introducción y objetivos
- Diagrama esquemático del circuito (Proteus)
- Código fuente comentado (.ino)
- Explicación del formato .org y ejemplo completo (archivo de prueba propio)
- Tabla de direcciones de EEPROM (cómo se almacenan los modos)
- Presupuesto estimado de componentes
- Fotos de la maqueta terminada e integrantes durante el desarrollo del proyecto
- Conclusiones técnicas y aprendizajes
- Anexos (los que crean necesarios)

11. Repositorio GitHub

- Nombre del repositorio: LABORGA_2S2025_G#/Proyecto
- Contenido obligatorio:
    - Archivo .ino
    - Imágenes del montaje físico
    - Diagramas del circuito
    - Ejemplos de archivos .org
    - Documentación en formato markdown

Consideraciones

Sincronización del sistema y retroalimentación visual

- Asegurarse de que los comandos enviados desde el dispositivo móvil o la PC sean procesados correctamente por el Arduino y que el estado del sistema se actualice de forma inmediata.
- Los LEDs de indicación (azul, verde, rojo) deben responder de manera clara y sincronizada al modo de operación:
    - LED azul: Encendido constante cuando el sistema está activo y listo para recibir comandos.
    - LED verde: Parpadeo breve al cargar exitosamente un archivo .org.
    - LED rojo: Encendido permanente o parpadeo lento si se detecta un error (archivo .org inválido o fallo en escritura de EEPROM).

- La pantalla LCD debe actualizar su contenido de forma inmediata tras cada cambio de modo, mostrando el nombre del ambiente activo y el estado del ventilador.
- Al encender o reiniciar el Arduino, la puerta debe cerrarse automáticamente, independientemente de su estado previo, sin necesidad de guardar su posición en memoria.

Uso de la memoria EEPROM

- Todos los modos predefinidos (fiesta, relajado, noche, encender_todo, apagar_todo) deben almacenarse en la EEPROM interna del Arduino para garantizar persistencia tras reinicios.
- Cada modo ocupa una dirección específica en la EEPROM (por ejemplo: dirección 0 para fiesta, dirección 10 para relajado, etc.). Se debe mantener una tabla clara de asignación de direcciones en la documentación.
- Antes de escribir en la EEPROM, validar que los datos no estén corruptos o fuera de rango.
- No se permitirá el uso de variables globales persistentes fuera de la EEPROM para simular almacenamiento; todo dato de configuración debe guardarse en esta memoria no volátil.

Interfaz de configuración y archivo .org

- Durante la carga del archivo vía puerto serial, el sistema debe ignorar comandos Bluetooth hasta que la operación termine.
- Si se detecta una línea con formato incorrecto (ej: salon:on, habitacion:abc), el proceso se detiene, se activa el LED rojo y se muestra en el LCD: "Error: Archivo .org".

Control del ventilador (motor DC)

- El motor DC simula un ventilador de techo, este debe girar únicamente en un sentido.
- Se controla directamente mediante una salida digital del Arduino, opcionalmente usando un transistor NPN (2N2222) como interruptor si el consumo de corriente lo requiere.
- El ventilador se activa o desactiva según el modo seleccionado.

Comunicación y control remoto

- El módulo Bluetooth permite recibir comandos desde una aplicación terminal en un dispositivo móvil.
- Comandos válidos:
    - modo_fiesta
    - modo_relajado
    - modo_noche
    - encender_todo
    - apagar_todo
    - estado

- El Arduino debe responder con un mensaje de confirmación vía Bluetooth (y en el LCD) tras cada comando exitoso.
- No se permite modificar los estados de las luces o el ventilador directamente desde el código sin pasar por un comando definido.

Uso del Arduino y componentes electrónicos

- Se permite el uso de Arduino (Uno, Nano o compatible) únicamente para:
    - Almacenar y gestionar los modos en EEPROM.
    - Procesar comandos recibidos por USB o Bluetooth.
    - Activar salidas digitales hacia LEDs y motor.
    - Manejar la pantalla LCD (I2C o paralelo).
    - Monitorear el estado del sistema y generar retroalimentación visual (LEDs, LCD).
- El Arduino no debe usarse para simular lógica secuencial con delays excesivos o bucles bloqueantes; se recomienda el uso de millis() para gestión temporal no bloqueante.
- Todos los componentes utilizados deben pertenecer a la lista oficial del curso o estar previamente autorizados.

Listado de componentes preaprovados.

DESCRIPCIÓN	CÓDIGO
Compuerta AND	7408
Compuerta NOT	7404
Compuerta XOR	7486
Compuerta OR	7432
Transistores NPN/PNP	2N2222A
Comparador	7485
Decoder	7447 / 7448
Multiplexor	74157
Bluetooth	HC-06/05
Pantalla LCD	I2C (16x2)
Motor DC	N/a
Push Button	N/a
Buzzer	N/a
Arduino (Uno, Mega, nano)	N/a

Requisitos Generales

- **Claridad del Circuito**: El diseño debe ser claro, ordenado y legible. Se aplicará penalización por desorden.
- **Implementación**: Se tomará en cuenta la presentación de la maqueta por lo que se tomará en cuenta la creatividad.

Cronograma

Tarea	Fecha
Asignación de la práctica / Entrega del enunciado	18/04/2026
Fecha de entrega	02/05/2026
Fecha de calificación	02/05/2026

Rúbrica de Calificación

Criterio	Descripción	Punteo
Matriz de Iluminación y Control por Ambientes	Sistema de control de luces por ambiente (sala, comedor, cocina, baño, habitación) mediante salidas digitales del Arduino. Funciona correctamente: encendido/apagado individual por ambiente y activación simultánea por escenas predefinidas. El estado de cada LED debe corresponder exactamente al modo seleccionado.	15
Gestión de Modos mediante EEPROM y Archivo .org	Implementación correcta del almacenamiento de modos en la EEPROM interna del Arduino. El archivo.org debe seguir el formato especificado. El sistema debe cargar correctamente el archivo vía puerto serial, validar su sintaxis y guardar los estados en EEPROM. Si hay error, debe mostrar mensaje en LCD y encender LED rojo.	15
Control Remoto vía Bluetooth	Implementación del módulo Bluetooth para recibir comandos desde dispositivo móvil (app terminal). Los comandos soportados. El Arduino debe interpretarlos correctamente y aplicar el modo correspondiente. La comunicación debe ser estable y sin pérdida de datos.	14
Pantalla LCD (I2C, 16x2) y Retroalimentación Visual	La pantalla LCD debe mostrar información precisa según el modo activo: nombre del modo, estado del ventilador y comportamiento de los LEDs. Debe actualizarse inmediatamente tras cada cambio. En caso de error, debe mostrar el mensaje de error correspondiente.	12
Indicadores visuales (LEDs) y displays	Uso de 3 LEDs para retroalimentación visual: -LED Azul: Encendido constante cuando el sistema está activo y listo para recibir comandos. -LED Verde: Parpadea brevemente (3 veces) al finalizar carga exitosa del archivo.orgo ejecución correcta de un comando. -LED Rojo: Encendido permanente si hay error en el archivo.org, comando inválido o fallo en escritura de EEPROM.	10
Control del Ventilador (Motor DC)	Motor DC girando únicamente. Se activa/desactiva mediante señal digital del Arduino (opcionalmente usando transistor NPN como interruptor).	6
Documentación técnica	Incluye carátula, introducción, funciones booleanas, mapas de Karnaugh (si aplica), diagramas del circuito (capturas del .pdsprj), lista de equipo, presupuesto estimado y conclusiones. Claridad, orden y profundidad técnica.	6
Maqueta física y funcional	Presentación clara y funcional del sistema. Circuito ordenado, incluye representación física de la casa inteligente. Creatividad, estabilidad del sistema y calidad de ensamblado.	10
Funcionamiento Integrado y Estabilidad del Sistema	Todos los módulos (maqueta, LEDs, motor, Bluetooth, LCD, EEPROM) deben funcionar de forma coordinada y estable. El sistema debe responder correctamente a comandos, mantener los estados guardados tras reinicio, y no bloquearse ni colapsar durante la operación. La sincronización entre modos, LEDs y ventilador debe ser precisa.	12

Valores

En el desarrollo de la práctica, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
    - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
    - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
    - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
3. **Uso Responsable de Recursos Externos**
    - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
4. **Revisión y Detección de Plagio**
    - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
    - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.

Referencias

- Floyd, T. L. (2015). Fundamentos de Sistemas Digitales. Pearson.
- Tocci, R., Widmer, N., & Moss, G. (2011). Sistemas Digitales: Principios y Aplicaciones. Pearson.
- Datasheets de CI 74LS74, 7485, 7447, Arduino.